기본 개념: zkSNARK와 STARK 이해하기

개요

이 문서는 영지식 증명(Zero-Knowledge Proof) 시스템, 특히 zkSNARK와 STARK의 기본 개념을 다룹니다. Plonky3를 이해하기 위한 필수적인 암호학적 배경 지식을 제공하며, 수학적 정의와 직관적 설명을 균형 있게 구성했습니다.

1. 영지식 증명(Zero-Knowledge Proof)이란?

1.1 정의와 핵심 특성

영지식 증명은 증명자(Prover)가 검증자(Verifier)에게 어떤 비밀 정보를 알고 있다는 사실을 증명하되, 그 비밀 정보 자체는 전혀 노출하지 않는 암호학적 프로토콜입니다.

영지식 증명이 만족해야 하는 세 가지 핵심 특성:

1. **완전성(Completeness)**: 참인 명제에 대해서는 정직한 증명자가 항상 검증자를 설득할 수 있어야 합니다.

2. **건전성(Soundness)**: 거짓인 명제에 대해서는 어떤 악의적인 증명자도 검증자를 속일 수 없어야 합니다.

3. **영지식성(Zero-Knowledge)**: 검증 과정에서 비밀 정보에 대한 어떤 추가적인 지식도 노출되지 않아야 합니다.

1.2 직관적 이해: 동굴의 비유

앨리(Alice)가 밥(Bob)에게 비밀 통로가 있는 동굴의 비밀번호를 알고 있다고 증명하려고 합니다:

1. 앨리스는 동굴 입구에서 무작위로 왼쪽 또는 오른쪽 길을 선택해 들어갑니다.

2. 밥은 앨리스가 어느 쪽으로 갔는지 모른 채, 무작위로 한쪽에서 나오라고 요청합니다.

3. 앨리스가 비밀번호를 안다면 항상 요청받은 쪽에서 나올 수 있습니다.

4. 이 과정을 여러 번 반복하면, 밥은 앨리스가 비밀번호를 안다고 확신할 수 있습니다.

이 과정에서 밥은 비밀번호 자체는 전혀 알 수 없습니다.

2. SNARK: Succinct Non-Interactive Arguments of Knowledge

2.1 SNARK의 특징

SNARK는 다음과 같은 특성을 가진 영지식 증명 시스템입니다:

- **간결성(Succinct)**: 증명의 크기가 원본 계산의 크기보다 훨씬 작습니다.
- **비상호작용성(Non-Interactive)**: 증명자와 검증자 간의 실시간 상호작용이 필요하지 않습니다.
- **논증(Argument)**: 계산적으로 제한된 적대자에 대해서만 건전성을 보장합니다.
- **지식(Knowledge)**: 증명자가 실제로 witness를 알고 있음을 보장합니다.

2.2 SNARK의 구성 요소

1. **회로(Circuit)**: 증명하고자 하는 계산을 산술 회로로 표현
2. **신뢰할 수 있는 설정(Trusted Setup)**: 증명과 검증에 필요한 공개 매개변수 생성
3. **증명 생성(Proving)**: witness와 공개 매개변수를 사용해 증명 생성
4. **검증(Verification)**: 공개 입력과 증명을 사용해 검증 수행

2.3 수학적 기초: 타원곡선과 페어링

SNARK는 주로 타원곡선 암호학과 bilinear pairing에 기반합니다:

$$e: G_1 \times G_2 \to G_t$$

여기서 e는 bilinear map이며, 다음 성질을 만족합니다:

- **Bilinearity**: e(aP, bQ) = e(P, Q)^(ab)
- **Non-degeneracy**: e(P, Q) ≠ 1 for some P, Q
- **Computability**: e(P, Q)를 효율적으로 계산 가능

3. STARK: Scalable Transparent Arguments of Knowledge

3.1 STARK의 특징

STARK는 SNARK의 한계를 극복하기 위해 개발된 시스템입니다:

- **확장성(Scalable)**: 증명 생성 시간이 계산 크기에 대해 준선형적으로 증가
- **투명성(Transparent)**: 신뢰할 수 있는 설정이 필요하지 않음
- **양자 저항성**: 양자 컴퓨터 공격에 대해 저항성을 가짐
- **간결성**: 증명 크기가 로그적으로 증가

3.2 STARK의 핵심 구성 요소

1. **AIR (Algebraic Intermediate Representation)**: 계산을 다항식 제약조건으로 표현
2. **FRI (Fast Reed-Solomon Interactive Oracle Proof)**: 다항식의 저차수성을 증명
3. **Merkle Tree**: 다항식 평가값들을 커밋
4. **Fiat-Shamir 변환**: 상호작용을 비상호작용으로 변환

3.3 수학적 기초: 유한체와 다항식

STARK는 유한체 위의 다항식 연산에 기반합니다:

```
F = GF(p) where p is a prime
```

핵심 아이디어:

- 계산을 다항식 P(x)로 인코딩
- P(x)가 특정 차수 이하임을 증명
- Reed-Solomon 코드를 사용해 오류 정정 능력 활용

4. SNARK vs STARK 비교

4.1 성능 비교

| 특성 | SNARK | STARK |

|------|-------|-------|

| 증명 크기 | 매우 작음 (~200 bytes) | 작음 (~100KB) |

| 증명 시간 | 빠름 | 매우 빠름 |

| 검증 시간 | 매우 빠름 | 빠름 |

| 설정 | 신뢰할 수 있는 설정 필요 | 투명한 설정 |

| 양자 저항성 | 없음 | 있음 |

4.2 보안 가정

SNARK:

- 이산 로그 문제의 어려움
- Knowledge of Exponent Assumption (KEA)
- 신뢰할 수 있는 설정의 무결성

STARK:

- 해시 함수의 충돌 저항성
- Reed-Solomon 코드의 거리 특성

- 다항식 저차수성 가정

4.3 사용 사례

SNARK가 적합한 경우:

- 증명 크기가 중요한 블록체인 애플리케이션
- 검증 비용을 최소화해야 하는 경우
- 설정 신뢰성을 보장할 수 있는 환경

STARK가 적합한 경우:

- 대규모 계산의 증명이 필요한 경우
- 신뢰할 수 있는 설정을 피하고 싶은 경우
- 양자 저항성이 필요한 장기적 보안

5. Polynomial IOP (Interactive Oracle Proof)

5.1 개념 소개

Polynomial IOP는 STARK의 이론적 기반이 되는 증명 시스템입니다. 증명자가 다항식들을 오라클로 제공하고, 검증자가 이 다항식들을 무작위 점에서 쿼리하여 검증을 수행합니다.

5.2 구성 요소

1. **다항식 오라클**: 증명자가 제공하는 다항식들
2. **쿼리 단계**: 검증자가 무작위 점에서 다항식 값을 요청
3. **일관성 검사**: 다항식들 간의 관계 검증
4. **저차수성 테스트**: 다항식이 주장된 차수 이하인지 확인

5.3 수학적 표현

다항식 P(x)에 대해:

- **차수 제한**: deg§ ≤ d
- **평가 일관성**: P® = v for random r
- **관계 만족**: $R(P_1, P_2, ..., P_k) = 0$

5.4 FRI와의 연결

FRI는 Polynomial IOP의 핵심 구성 요소로, 다항식의 저차수성을 효율적으로 증명합니다:

1. **폴딩(Folding)**: P(x) → P'(x) where deg(P') ≈ deg§/2
2. **재귀적 적용**: 차수가 충분히 작아질 때까지 반복
3. **최종 검증**: 작은 차수의 다항식을 직접 검증

6. Fiat-Shamir 변환

6.1 상호작용에서 비상호작용으로

Fiat-Shamir 변환은 상호작용 증명을 비상호작용 증명으로 변환하는 기법입니다:

1. **챌린지 생성**: 검증자의 무작위 챌린지를 해시 함수로 대체
2. **결정론적 과정**: 증명 과정이 완전히 결정론적이 됨
3. **보안 보장**: Random Oracle Model에서 보안성 유지

6.2 구현 방식

```
challenge = Hash(transcript || previous_messages)
```

여기서:

- `transcript` : 지금까지의 모든 메시지

- `Hash` : 암호학적 해시 함수 (예: SHA-256, Poseidon)

7. Plonky3에서의 적용

7.1 STARK 기반 설계

Plonky3는 STARK 기반의 증명 시스템으로, 다음과 같은 특징을 가집니다:

- **모듈형 설계**: 각 구성 요소를 독립적으로 교체 가능
- **다양한 유한체 지원**: Mersenne31, BabyBear, Goldilocks 등
- **최적화된 구현**: SIMD 명령어와 병렬 처리 활용
- **투명한 설정**: 신뢰할 수 있는 설정 불필요

7.2 핵심 혁신

1. **Circle STARK**: 원군(Circle Group)을 사용한 효율적인 FFT
2. **모듈형 AIR**: 다양한 계산을 표현할 수 있는 유연한 인터페이스
3. **최적화된 FRI**: 메모리와 계산 효율성을 극대화한 구현

결론

영지식 증명 기술은 프라이버시와 확장성을 동시에 제공하는 혁신적인 암호학 도구입니다. SNARK와 STARK는 각각 다른 장단점을 가지며, 사용 사례에 따라 적절한 선택이 필요합니다.

Plonky3는 STARK의 장점을 극대화하면서도 실용적인 성능을 제공하는 현대적인 구현체로, 차세대 영지식 증명 애플리케이션의 기반이 될 것입니다.

다음 문서에서는 이러한 개념들을 뒷받침하는 수학적 기초에 대해 더 자세히 살펴보겠습니다.